The Globe oposiciones

Bloque 3 - Tema 6

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS

Resumen C

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE 2

1. INTRODUCCIÓN 3
   1. *Palabras reservadas del lenguaje 3*
   2. *Identificadores 3*
   3. *Variables, tipos de datos y constantes 3*
2. OPERADORES 8
3. ESTRUCTURA DE UN PROGRAMA EN C 11
4. SENTENCIAS DE CONTROL Y BUCLES 12
   1. *Sentencias if 12*
   2. *Sentencias switch 12*
   3. *Bucle for 13*
   4. *Bucle while 13*
   5. *Bucle do/while 13*
   6. *Sentencias break y continue 13*
5. ARRAYS, CADENAS Y PUNTEROS 15
   1. *Arrays y cadenas 15*
   2. *Punteros 15*
6. FUNCIONES 18
   1. *Paso de parámetros 18*
   2. *Función main 19*
7. ESTRUCTURAS Y ENUMERACIONES 20
8. ENTRADA / SALIDA 22
9. EL PREPROCESADOR 25

1. INTRODUCCIÓN

El lenguaje de programación C fue creado por Brian Kernighan y Dennis Ritchie a mediados de los años 70. En un lenguaje de programación estructurado.

1. Palabras reservadas del lenguaje

El lenguaje C posee un número reducido de palabras reservadas (32) que define el estándar ANSI-C:

PALABRAS RESERVADAS							
auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

Las palabras reservadas se escriben siempre minúsculas.

2. Identificadores

C es un lenguaje sensible al contexto, diferencia entre mayúsculas y minúsculas.

Un **identificador** es cualquier palabra no reservada que comience por una letra o por un subrayado, pudiendo contener en su interior letras, números y subrayados. La longitud máxima de un identificador depende del compilador usado, generalmente, suelen ser de 32 caracteres.

Son usados para nombrar a varios elementos de un programa, como variables, funciones y arrays.

Ejemplo:

Identificadores correctos: `x`, `_y`, `y22`, `VARIABLE`, `tanto_por_ciento`
Identificadores erróneos: 5d, “44, por-ciento, por ciento

3. Variables, tipos de datos y constantes

En C todas las variables han de ser declaradas antes de ser utilizadas. En concreto, las variables tienen que ser declaradas al comienzo de un bloque, antes de la primera sentencia ejecutable de dicho bloque. La declaración tiene el siguiente formato:

```
<tipo_dato> <nombre_variable> [, <nombre_variable];
```

TIPO	TAMAÑO	MÍNIMO	MÁXIMO
unsigned char	8 bits	0	255
char	8 bits	-128	127
enum	16 bits	-32768	32767
int	16 bits	-32768	32767
unsigned int	16 bits	0	65535
short int	16 bits	-32768	32767
unsigned long	32 bits	0	4294967295
long	32 bits	-2147483648	2147483647
float	32 bits	$-3{,}4*10^{-38}$	$3{,}4*10^{38}$
double	64 bits	$-1{,}7*10^{-308}$	$1{,}7*10^{308}$
long double	64 u 80 bits	$-1{,}7*10^{-308}$	$1{,}7*10^{308}$

Ejemplos: `int a=4;` `char asterisco='*';` `float suma=0.11;`

Los modificadores de acceso limitan el uso que puede realizarse de las variables declaradas. Los modificadores de acceso anteceden a la declaración del tipo de dato de la variable y son los siguientes:

MODIFICADOR	EFECTO
const	Variable de valor constante. Solo puede ser modificado en el momento de la declaración, después error de compilación. Ejemplo: `const int x=237;`
volatile	Variable cuyo valor es modificado externamente por otro proceso (como la hora del sistema). Fuerza a que cada vez que se usa la variable se realice una comprobación de su valor. Ejemplo: `volatile const int hora;`

En C, las variables pueden ser declaradas en cuatro lugares del módulo del programa:

- Fuera de todas las funciones del programa, son las llamadas **variables globales**, accesibles desde cualquier parte del programa.
- Dentro de una función, son las llamadas **variables locales**, accesibles tan solo por la función en las que se declaran.
- Como **parámetros a la función**, accesibles de igual forma que si se declararan dentro de la función.

- Dentro de un bloque de código del programa, accesible tan solo dentro del bloque donde se declara. Esta forma de declaración puede interpretarse como una **variable local del bloque** donde se declara.

Ejemplo:

```
#include <stdio.h>
int sum; /* Variable global, accesible desde cualquier parte */
            /* del programa*/
void suma(int x) /* Variable local declarada como parámetro, */
                    /* accesible solo por la función suma*/
{
      sum=sum+x;
      return;
}
void intercambio(int *a,int *b)
{
      if (*a>*b)
      {
            int inter; /* Variable local, accesible solo dentro del */
                        /* bloque donde se declara*/
            inter=*a;
            *a=*b;
            *b=inter;
      }
      return;
}
int main(void) /*Función principal del programa*/
{
      int contador,a=9,b=0; /*Variables locales, accesibles solo */
                              /* por main*/
      sum=0;
      intercambio(&a,&b);
      for(contador=a;contador<=b;contador++) suma(contador);
      printf("%d\n",suma);
      return(0);
}
```

En C, las **constantes** se refieren a los valores fijos que el programa no puede alterar. Algunos ejemplos de constantes de C son:

TIPO DE DATO	EJEMPLO		
char	'a'	'9'	'Q'
int	1	-34	21000
long int	-34	6856L	456
short int	10	-12	1500
unsigned int	4560U	345	3
float	12.45	4.3e-3	-2.8e9
double	-34.6	-2.2e7	1.0e10

Ejemplos:

Constantes hexadecimales: 0x34, 0xFFFF
Constantes en base octal: 011, 0173
Constantes cadenas de caracteres: “cadena encerrada entre comillas dobles”.

Ciertos caracteres no imprimibles, como las comillas dobles ("), el apóstrofo o comilla simple ('), etc. Se pueden expresar en términos de una **secuencia de escape**:

CÓDIGO	SIGNIFICADO
\b	retroceso (backspace)
\f	avance de página
\n	nueva línea
\r	retorno de carro
\t	tabulador horizontal
\"	comilla doble
\'	comilla simple
\0	nulo
****	barra invertida
\v	tabulador vertical
\a	alarma
\o	constante octal
\x	constante hexadecimal

Conversión AUTOMÁTICA de tipos

El lenguaje C permite que en una misma expresión aparezcan diferentes tipos de datos, encargándose el compilador de realizar las operaciones de forma correcta. El compilador del lenguaje C, cuando en una misma expresión aparecen dos o más tipos de datos, convierte todos los operandos al tipo del operando más grande existente de acuerdo con las dos reglas siguientes:

- Todos los `char` y `short int` se convierten a `int`. Todos los `float` a `double`.
- Para todo par de operandos, lo siguiente ocurre en secuencia:
  - Si uno de los operandos es un `long` double, el otro se convierte en `long double`.
  - Si uno de los operandos es `double`, el otro se convierte a `double`.
  - Si uno de los operandos es `long`, el otro se convierte a `long`.
  - Si uno de los operandos es `unsigned`, el otro se convierte a `unsigned`.

Conversión EXPLÍCITA de tipos

En C, existe la posibilidad de forzar la conversión de un tipo de datos en otro tipo de datos. Esta conversión de un tipo de datos en otro se llama "casting", y su sintaxis es:

```
(tipo)expresión
```

Ejemplo:

```
int a=3, b=2;
float c;
c=a/b;
```

La operación asigna a c el valor 1.0 en vez del valor 1.5, porque a y b son variables de tipo entero. Se realiza una división entre enteros, y el resultado de 3/2 es 1. A continuación ese valor 1 se convierte a un valor en coma flotante para realizar la asignación (valor 1.0), y se asigna a c. Si lo que se desea es que la división se realice en punto flotante, debe escribirse la operación de la siguiente forma:

```
c=(float)a/b;
```

Esta conversión obliga a convertir la variable a en `float`, y, aplicando las reglas de conversión automática de tipos, se realiza la división en coma flotante, dando lugar a que el resultado de la operación sea 1.5, y dicho valor sea el asignado a la variable c.

2. OPERADORES

TIPO	OPERADOR	SIGNIFICADO
Aritméticos	**+**	Adición
	-	sustracción
	*****	multiplicación
	/	división
	%	resto de división entera (solo int, char)
Lógicos y relacionales	**>**	mayor que
	>=	mayor o igual que
	<	menor que
	<=	menor o igual que
	==	igual
	!=	distinto
	&&	AND lógico
	||	OR lógico
	!	NOT lógico
Asignación	**++**	incremento unario (solo int, char)
	--	decremento unario (solo int, char)
	=	asignación simple
	***=**	asignación de la multiplicación
	/=	asignación de la división
	%=	asignación del resto
	+=	asignación de la suma
	-=	asignación de la resta
	<<=	asignación del desplazam. a la izquierda
	>>=	asignación del desplazam. a la derecha
	&=	asignación de la operación AND
	|=	asignación de la operación OR

	^=	asignación de la operación XOR
Tratamiento de bits	**~**	operación NOT
	&	operación AND
	|	operación OR
	^	operación XOR
	>>	desplazam. a la derecha
	<<	desplazam. a la izquierda
Punteros	*****	valor de la variable a la que apunta el operando
	&	dirección del operando
Acceso a estructuras y uniones	**->**	acceso a los campos si puntero apunta a estructura
	.	acceso a los campos si no hay puntero
Matrices	**[]**	acceso a los elementos de una matriz
Expresiones	**()**	Modificación orden evaluación expresiones
Tamaño	**sizeof**	Tamaño de una variable
Condicional	**?**	Condición ? true : false

PRECEDENCIA OPERADORES
() [] -> .
! ~ ++ --
- * &
sizeof
* / %
+ -
<< >>
< <= >= >
== !=
&
^
|

&&
||
?:
= += -= *= /= %=
&= ^= |= <<= >>=

3. ESTRUCTURA DE UN PROGRAMA EN C

Todo programa de C consta, básicamente, de un conjunto de funciones, y una función llamada **`main`**, la cual es la primera que se ejecuta al comenzar el programa, llamándose desde ella al resto de funciones que compongan nuestro programa. Cada función debe contener:

- Una **cabecera** de la función, que consta del nombre de la función, seguido de una lista opcional de argumentos encerrados con paréntesis.
- Una lista de **declaración** de argumentos, si se incluyen estos en la cabecera.
- Una **sentencia compuesta**, que contiene el resto de la función. Cada sentencia compuesta se encierra con un par de llaves, {.....}. Las llaves pueden contener combinaciones de sentencias elementales (denominadas sentencias de expresión) y otras sentencias compuestas. Así las sentencias compuestas pueden estar anidadas, una dentro de otra. Cada sentencia de expresión debe acabar en punto y coma (;).

Los comentarios pueden aparecer en cualquier parte del programa, mientras estén situados entre los delimitadores `/* ..... */` (por ejemplo: `/*esto es un ejemplo*/`). Los comentarios son útiles para identificar los elementos principales de un programa o simplemente para orientar a un posible usuario de ese código.

```
#include <stdio.h>

main()
{
int cont;
for(cont=1;cont <= 3; cont++) {  /* Inicializo bucle en la 1ª vuelta a cont=1,*/
    printf("Iteración %d\t", cont);/* incrementa en cada vuelta el cont a 1
(cont++)*/
    incrementa();     /* repito el cuerpo del bucle mientras cont<=3 */
}  /* fin del for */

/*  El %d del printf me indica que voy a representar por pantalla un número entero
cont, si en lugar de ser cont de tipo entero, fuera de tipo float, y pusiese en el
printf un %d, me escribiría por pantalla su parte entera (todo esto de la entrada y
salida se verá mejor en el capítulo siguiente).*/
} /* fin del main() (programa principal)*/

void incrementa(){
int a=1;
int b=1;

printf("a=%d b=%d\n", a,b);
a++;b++;
}  /* fin de la función incrementa */
```

La salida de este programa es:

Iteración 1 → a=1 b=1
Iteración 2 → a=1 b=1
Iteración 3 → a=1 b=1

4. SENTENCIAS DE CONTROL Y BUCLES

1. Sentencias if

La forma general de la sentencia `if` es:

```
if (condición)
	sentencia;
else
	sentencia;
```

`else` es opcional. Si la condición es verdadera se ejecuta la sentencia asociada al `if`, en caso de que sea falsa la condición se ejecuta la sentencia asociada al `else` (si existe el `else`).

Ejercicio: Valor de a y b al terminar la ejecución del código

```
int a,b;
if (a>b)
{
	b--;
	a=a+5;
}
else
{
	a++;
	b=b-5;
}
if (b-a!=7)
	b=5;
```

If anidados: C asocia cada else al `if` más cercano posible y que todavía no tiene ningún `else` asociado.

2. Sentencias switch

La forma general de la sentencia switch es:

```
switch(variable)
{
	case const1:
		sentencia;
		break;
	case const2:
		sentencia;
		break;
	...
	default:
		sentencia;
}
```

Donde variable debe ser de tipo `char` o `int`, y donde `const1`, `const2`, ..., indican constantes de C del tipo de datos de la variable. Dichas constantes no pueden repetirse dentro del `switch`. El `default` es opcional y puede no aparecer, así como los `break` de los `case`. La sentencia `switch` se ejecuta comparando el valor de la variable con el valor de cada una de las constantes,

realizando la comparación desde arriba hacia abajo. En caso de que se encuentre una constante cuyo valor coincida con el valor de la variable, se empieza a ejecutar las sentencias hasta encontrar una sentencia `break`. En caso de que no se encuentre ningún valor que coincida, se ejecuta el `default` (si existe).

```
char d;
int cont=0;
switch(d)
{
	case ‘\r’: cont++;  /* Si d es un retorno de carro, se */
				/* ejecuta este cont++ y el siguiente*/
				/* al no aparecer un break */
	case ‘\x1B’: cont++;
			break;
	default: cont=-1;
}
```

3. Bucle for

La sintaxis del bucle for es:

```
for(inicialización; condición; incremento) sentencia;
```

4. Bucle while

La sintaxis del bucle while es:

```
while (condición) sentencia;
```

Donde la sentencia puede no existir (sentencia vacía), pero siempre debe existir la condición.

5. Bucle do/while

La sintaxis del bucle do/while es:

```
do
	sentencia;
while (condición);
```

6. Sentencias break y continue

Las sentencias de control `break` y `continue` permiten modificar y controlar la ejecución de los bucles anteriormente descritos.

La sentencia **`break`** provoca la salida del bucle en el cual se encuentra y la ejecución de la sentencia que se encuentra a continuación del bucle.

La sentencia **`continue`** provoca que el programa vaya directamente a comprobar la condición del bucle en los bucles `while` y `do/while`, o bien, que ejecute el incremento y después compruebe la condición en el caso del bucle `for`.

5. ARRAYS, CADENAS Y PUNTEROS

1. Arrays y cadenas

Un **array unidimensional** se declara como:

```
tipo nombre[tamaño];
```

El primer elemento de un array es el que posee el índice 0 y el último, tamaño-1.

Una **cadena**, también llamada string, es un tipo especial de array unidimensional. Una cadena es un array de caracteres (`char`) que termina con un carácter especial (el carácter '\0'). Es por ello, que la declaración de una cadena de caracteres se realiza exactamente igual que la declaración de un array unidimensional de caracteres:

```
char cadena[tamaño];
```

La declaración de arrays de más de una dimensión se realiza de forma parecida a la de una dimensión, la sintaxis de la declaración de un **array multidimensional** es:

```
tipo nombre[tam1][tam2]...[tamN];
```

Además, es posible inicializar los arrays en el momento de declararlos. Su sintaxis es:

```
tipo nombre[tam1][tam2]...[tamN]={lista_de_valores};
```

Ejemplo: `float vector[3]={-3.0,5.7,-7.5};`

También es posible inicializar arrays sin ponerles el tamaño, el compilador cuenta el número de caracteres de inicialización y reserva el tamaño necesario de forma automática.

Ejemplo: `float vector[]={-3.0,5.7,-7.5};` `char cadena[]="Esto es una cadena";`

2. Punteros

Los punteros son una de las poderosas herramientas que ofrece el lenguaje C a los programadores, pero también es una fuente de errores.

Un **puntero** es una variable que contiene una dirección de memoria. Normalmente esa dirección es una posición de memoria de otra variable, por lo cual se suele decir que el puntero "apunta" a la otra variable. La sintaxis de la declaración de una variable puntero es:

```
tipo *nombre;
```

El tipo base de la declaración sirve para conocer el tipo de datos al que pertenece la variable a la cual apunta la variable de tipo puntero. Esto es fundamental para poder leer el valor que almacena la zona de memoria apuntada por la variable puntero y para poder realizar ciertas operaciones aritméticas sobre los mismos.

Ejemplos: `int *a;` `char *p;` `float *f;`

Existen dos operadores especiales de los punteros, el **operador *** y el **operador &**.

El **operador &** es un operador unario que devuelve la dirección de una variable de memoria.

Ejemplo: Declaramos `int *a,b;`

Después: `a=&b;` La variable puntero `a` contendrá la dirección de memoria de la variable `b`.

El **operador *** es un operador unario que devuelve el valor de la variable situada en la dirección que sigue.

Ejemplo: `int *a,b,c;`

```
b=15;
a=&b;
c=*a;
```

Entonces la variable `c` contendrá el valor `15`, pues `*a` devuelve el valor de la dirección que sigue (a la que "apunta" la variable puntero), y con anterioridad hemos hecho que `a` contenga la dirección de memoria de la variable `b` usando para ello el operador `&`.

Asignación de punteros

Es posible asignar el valor de una variable de tipo puntero a otra variable de tipo puntero.

Ejemplo: `int *a,*b,c;` `a=&c;` `b=a;`

`b` contiene el valor de `a`, y por ello, `b` también "apunta" a la variable `c`.

Aritmética de punteros

Sobre las variables de tipo puntero es posible utilizar los operadores `+`, `-`, `++` y `--`. Estos operadores incrementan o decrementan la posición de memoria a la que "apunta" la variable puntero. El incremento o decremento se realiza de acuerdo al tipo base de la variable de tipo puntero, de ahí la importancia del tipo del que se declara la variable puntero.

Ejemplo: `tipo *a;` `a=a+num;`

La posición a la que apunta `a` se incrementa en `num*sizeof(tipo)`. Para la resta se decrementa de igual forma en `num*sizeof(tipo)`. Los operadores `++` y `--` son equivalentes a realizar `num=1`.

Comparación de punteros

Sobre las variables de tipo puntero es posible realizar operaciones de comparación entre ellas.

Ejemplo:

```
int *a,*b;
if (a<b)
   printf("a apunta a una dirección más baja que b");
```

Relación entre punteros y arrays

Partimos del siguiente ejemplo:

```
char str[80], *p;          p=str;
```

Este fragmento de código asigna en la variable puntero p la dirección del primer elemento del array `str`. Entonces, es posible acceder al valor de la quinta posición del array mediante `str[4]` y `*(p+4)` (el índice comienza en 0). Esta estrecha relación entre los arrays y los punteros queda más evidente si se tiene en cuenta que el nombre del array sin índice es la dirección de comienzo del array y, si además, se tiene en cuenta que un puntero puede indexarse como un array unidimensional, por lo cual, en el ejemplo anterior, podríamos referenciar ese elemento como `p[4]`.

Ejemplo: `p=&str[4];`

Entonces, el puntero `p` contiene la dirección del quinto elemento del array `str`.

Es posible, como con cualquier otro tipo de datos, definir un array de variables puntero. La declaración para un array de punteros int de tamaño 10 es: `int *a[10];`

Para asignar una dirección de una variable entera, llamada `var`, al tercer elemento del array de punteros, se escribe: `x[2]=&var;`

Y para encontrar el valor de `var`: `*x[2];`

Por último, como un puntero es también una variable, podemos definir un **puntero a puntero**.

Ejemplo: `int a, *b, **c;          b=&a;          c=&b;`

Entonces, `**c` tiene el valor de la variable `b`, pues `c` es un puntero a una variable que ya es de tipo puntero. Este concepto de puntero a puntero podría extenderse a puntero a puntero a puntero, etc.

6. FUNCIONES

El formato general de una función de C es:

```
tipo nombre(lista de parámetros)
{
	cuerpo de la función
}
```

La **sentencia `return`** permite, en primer lugar, salir de la función desde cualquier punto de la misma, y en segundo lugar, devolver un valor del tipo de la función, si ello es necesario (no se devuelve ningún valor si la función es de tipo `void`).

```
int Comparacion(int a,int b)
{
if (a>b) return 1;  /* a es mayor que b */
if (a<b) return -1;       /* a es menor que b */
return 0;                 /* a y b son iguales */
}
```

1. Paso de parámetros

El lenguaje C, solo permite pasar parámetros a las funciones por valor. Si se desea que los cambios efectuados en una función sobre una variable afecten fuera del alcance de la función, es posible simular un paso por referencia mediante el uso de punteros. En efecto, si a una función le pasamos como argumento la dirección de una variable, cualquier modificación que se realice en esa dirección, afectara, lógicamente, al valor que tiene la variable original, y con ello, conseguiremos el mismo efecto que un paso por referencia.

Ejemplo:

```
#include <stdio.h>
void Alfa(int *val,float pos)
{
	*val=5;
	pos=7.7;
	return;
}
void Beta(int val,float *pos)
{
	val=10;
	*pos=14.7;
}
int main(void)
{
	int a=6;
	float b=9.87;
	printf("Al principio valen a=%d b=%f\n",a,b);
	Alfa(&a,b);
	printf("Después de Alfa valen a=%d b=%f\n",a,b);
	Beta(a,&b);
	printf("Después de Beta valen a=%d b=%f\n",a,b);
}
```

Al principio, a=6 y b=9.87.
Después de Alfa, a=5 y b=9.87.
Después de Beta, a=5 y b=14.7.
A Alfa se le pasa la variable a por "referencia" (se le pasa &a, o sea, un puntero a la variable a), y la variable b por valor, mientras que en Beta sucede al revés.

2. Función main

La función main(), como toda función de C, acepta argumentos:

```
int main(int argc,char *argv[],char *env[])
```

Los argumentos son un entero (int argc), un array de punteros a strings (char *argv[]), y otro array de punteros a strings (char *env[]). El significado de los parámetros argc, argv y env es el siguiente:

- El parámetro **argc** contiene el número de argumentos en la línea de ordenes de la llamada al programa.
- El parámetro **argv** contiene un puntero a cada uno de los argumentos (strings) de la línea de ordenes de la llamada al programa.
- El parámetro **env** contiene un puntero a cada una de las variables de entorno (strings) del sistema operativo.

Ejemplo:

```
#include <stdio.h>
int main(int argc,char *argv[],char *env[])
{
	int i;
	printf("El valor de argc es: %d\n",argc);
	for(i=0;i<argc;i++)
		printf("El argumento %d es: %s\n",i,argv[i]);
	for(i=0;env[i]!=NULL;i++)
		printf("La variable de ambiente %d es: %s\n",i,env[i]);
	return 0;
}
```

Supongamos que el programa lo hemos llamado aplicacion.exe, entonces, llamando al programa con la siguiente línea:

```
aplicacion.exe argumento1 argumento2
```

Resultado:

El valor de argc es: 3
El argumento 0 es: aplicacion.exe
El argumento 1 es: argumento1
El argumento 2 es: argumento2
La variable de entorno 0 es: COMSPEC=C:\DOS\COMMAND.COM
La variable de entorno1 es: TEMP=C:\WINDOWS\TEMP
La variable de entorno 2 es: PROMPT=$HOME

7. ESTRUCTURAS Y ENUMERACIONES

Una **estructura** es un conjunto de variables que se referencian bajo el mismo nombre. La sintaxis de la declaración de una estructura en lenguaje C es:

```
struct nombre_estructura{
	tipo nombre_variable;
	tipo nombre_variable;
	...
	tipo nombre_variable;
}variables_estructura;
```

Es posible no poner el nombre de la estructura (`nombre_estructura`), o bien, no crear en el momento de declarar la estructura ninguna variable de la estructura (`variables_estructura`), pero no es posible eliminar la aparición de ambos elementos.

Ejemplo:

```
struct LISTA{
	int tam;
	char cadena[50];
}var_lista;

struct nodo{
	int tam;
	float vector[3];
	struct nodo *siguiente;
} n;
```

Para referenciar un elemento de una estructura se realiza de la siguiente forma:

```
variables_estructura.nombre_variable;
```

Ejemplo: `var_lista.tam; var_list.cadena[7]; n.tam;`

Una **enumeración** es un conjunto de constantes enteras con nombre y especifica todos los valores legales que pueden tener unas variables. Las enumeraciones se declaran de la siguiente forma:

```
enum nombre_enum{lista_de_enumeración} lista_de_variables;
```

Donde, al igual que en las estructuras, puede no aparecer `nombre_enum` o `lista_de_variables`.

Ejemplo:

```
enum DIAS{lunes, martes, miercoles, jueves, viernes } dias_semana;
```

Las enumeraciones asignan una constante entera a cada uno de los símbolos de la enumeración, empezando por el valor `0`. Esto puede modificarse colocando en la declaración el valor que deseamos tengan los elementos a partir de uno dado.

Ejemplo:

```
enum DIAS{lunes, martes=3, miercoles, jueves=100, viernes } dias_semana;
```

lunes=0, martes=3, miercoles=4, jueves=100 y viernes=101.

typedef

El lenguaje C permite mediante el uso de la palabra reservada **typedef** definir nuevos nombres para los tipos de datos existentes, esto no debe confundirse con la creación de un nuevo tipo de datos. La palabra clave `typedef` permite solo asignarle un nuevo nombre a un tipo de datos ya existente. La sintaxis general de uso de `typedef` es:

```
typedef tipo nombre;
```

Ejemplo:

```
typedef int entero;
typedef struct{
      unsigned codigo;
      char nombre[40];
      char apellido[40];
}cliente;
```

Creación de nuevas variables: `entero a;` `cliente b,*c;`

8. ENTRADA / SALIDA

Para el correcto funcionamiento de la entrada y salida en C se encuentran declaradas en el archivo de cabecera <**stdio.h**>, que es necesario incluir dicho archivo, mediante la directiva del preprocesador #include, para que la E/S funcione correctamente:

```
#include <stdio.h>
```

El lenguaje C tiene un extenso conjunto de funciones de biblioteca que ayudan a la hora de introducir, procesar y mostrar los datos. La biblioteca `<stdio.h>` incluye una serie de funciones de entrada/salida:

- `int getchar(void);`

  Lee un carácter desde la entrada estándar. Dicha función lee caracteres, de uno en uno, desde el teclado, esperando, para leer los caracteres, la pulsación de un retorno de carro. Por lo que es posible escribir varios caracteres antes de que se ninguno de ellos sea leído. La función `getchar()` hace eco en pantalla del carácter leído.

- `int putchar(int c);`

  Escribe un carácter en la salida estándar. Si sucede de forma correcta, devuelve el carácter escrito.

- `char *gets(char *s);`

  Lee un string desde la entrada estándar hasta que se pulsa un retorno de carro. El string es almacenado en la variable `s`, y el retorno de carro leído desde el teclado es, automáticamente, reemplazado por un carácter de final de string ('\0'). Devuelve un puntero a la variable `s` si sucede de forma correcta, y NULL en caso contrario. La función `gets()` permite corregir errores de teclado usando la tecla de retroceso antes de pulsar el retorno de carro.

- `int puts(const char *s);`

  Escribe un string en la salida estándar. El string es el almacenado en `s` y añade al final un retorno de carro.

- `int scanf(const char *formato[,dirección,...]);`

  Lee cualquier tipo de dato predefinido desde la entrada, y lo convierte de forma automática, al formato interno adecuado. El argumento formato es la "cadena de control" y el resto la "lista de argumentos".

  La cadena de control hace referencia a una cadena de caracteres que contiene información sobre el formato de los datos (debe haber tantos códigos de formato como argumentos), y la lista de argumentos son los argumentos que representan los datos.

  En la cadena de control se incluyen grupos individuales de caracteres cada uno de los cuales deben comenzar por %. Normalmente, un grupo de caracteres estará formado por el signo de porcentaje seguido de un carácter de conversión que indica el tipo del dato correspondiente.

ESPECIFICADOR	DESCRIPCIÓN
%c	Leer un único carácter
%d	Leer un entero decimal
%i	Leer un entero decimal
%e	Leer un número en punto flotante
%f	Leer un número en punto flotante
%g	Leer un número en punto flotante
%o	Leer un número octal
%s	Leer una cadena de caracteres
%x	Leer un número hexadecimal
%p	Leer un puntero
%n	Número de caracteres leídos
%u	Leer un entero sin signo

Dentro de la cadena de control se pueden encontrar varios caracteres seguidos o separados por espacios en blanco, tabuladores o caracteres de nueva línea (es muy frecuente el uso de caracteres en blanco para separar grupos de caracteres). El uso de caracteres de espaciado hace que `scanf` lea, pero no guarde, cualquier número de espacios en blanco hasta el primer carácter no blanco.

Ejemplo: `int x,y;` `scanf("%d% %d",&x,&y);`

NOTA: Los argumentos de `scanf` han de ser punteros que no son otra cosa que direcciones de memoria donde se encuentran los datos. Para obtener la dirección de memoria de una variable, es necesario aplicarle el operador unario & de la forma: `&variable`

- `int printf(const char *formato[,argumento,...]);`

Escribe cualquier tipo de dato en la salida estándar. El argumento formato es la "cadena de control", que son los caracteres que se envían a la salida estándar y los formatos que describen la forma en que se mostrarán los argumentos de la lista. Los formatos pueden ser:

ESPECIFICADOR	DESCRIPCIÓN
%c	Carácter
%d	Enteros decimales con signo
%i	Enteros decimales sin signo
%e	Punto flotante en notación científica (e minúscula)

%E	Punto flotante en notación científica (e mayúscula)
%f	Punto flotante
%o	Octal sin signo
%s	Cadena de caracteres
%x	Hexadecimal sin signo (letras minúsculas)
%X	Hexadecimal sin signo (letras mayúsculas)
%p	Puntero
%n	Número de caracteres escritos
%%	Imprimir el signo %

Recordad que en el argumento cadena de formato se pueden incluir secuencias de escape.

El resto de los argumentos se conoce como la "lista de argumentos", que representan los datos de salida.

Ejemplos:

```
#include <stdio.h>
int main(void)
{
	int a,b;
	printf("\nIntroduce el valor de a: ");
	scanf("%d",&a);
	printf("\nIntroduce el valor de b: ");
	scanf("%d",&b);
	if (b!=0)
		printf("\nEl valor de %d dividido %d es: %f\n",a,b,(float)a/b);
	else
		printf("\nError, b vale 0\n");
return 0;
}
```

```
#include <stdio.h>
int main()
{
	int fahr, celsius;
	int lower, upper, step;
	lower=0;
	upper=300;
	step=20;
	fahr=lower;
	while(fahr<=upper)
	{
		celsius=5*(fahr-32)/9;
		printf("%d\t%d\n",fahr,celsius);
		fahr=fahr+step;
	}
	return 0;
}
```

9. EL PREPROCESADOR

En un programa escrito en C, es posible incluir diversas instrucciones para el compilador dentro del código fuente del programa. Estas instrucciones dadas al compilador son llamadas directivas del preprocesador y, aunque realmente no son parte del lenguaje C, expanden el ámbito del entorno de programación de C.

El preprocesador, definido por el standard ANSI de C, contiene las siguientes directivas:

`#if`	`#ifdef`	`#ifndef`	`#else`
`#elif`	`#endif`	`#include`	`#define`
`#undef`	`#line`	`#error`	`#pragma`

Directiva #include

La directiva `#include` fuerza al compilador a incluir otro archivo fuente en el archivo que tiene la directiva `#include`, y a compilarlo. El nombre del archivo fuente a incluir se colocará entre comillas dobles o entre paréntesis de ángulo.

Ejemplo: `#include <stdio.h>` `#include "math.h"`

Los archivos incluidos mediante #include pueden a su vez poseer otras directivas `#include`. La diferencia existente entre encerrar el archivo entre paréntesis de ángulo o entre comillas dobles, es que, en el primer caso, se busca el archivo en los directorios de la línea de ordenes de compilación, y, después en los directorios estándar de C, pero nunca en el directorio de trabajo; y en el segundo caso, el primer sitio donde se busca el archivo a incluir es en el directorio actual de trabajo, pasándose, caso de no haber sido encontrado, a buscar en los mismos sitios que el caso anterior.

Directiva #define

La directiva define se usa para definir un identificador y una cadena que el compilador sustituirá por el identificador cada vez que se encuentre en el archivo fuente. El estándar ANSI llama al identificador "nombre de macro" y al proceso de sustitución "sustitución de macro".

Ejemplo: `#define TRUE 1` `#define FALSE 0`

El compilador, cada vez que vea el identificador `TRUE`, lo sustituirá por el valor `1`, y `FALSE` y lo sustituirá por `0`.

El uso más común de la directiva `#define` es la definición de valores constantes en el programa, tamaños de arrays, etc.

Ejemplo:

```
#include <stdio.h>
#define UPPER 300
```

```
#define LOWER 0
#define STEP 20

int main()
{
	int fahr, celsius;
	fahr=LOWER;
	while(fahr<=UPPER)
	{
		celsius=5*(fahr-32)/9;
		printf("%d\t%d\n",fahr,celsius);
		fahr=fahr+STEP;
	}
	return 0;
}
```